

Your Questions from the Class 10 Slip

Queues, pointers, circular buffers, and Valgrind

CSCI 235 – Data Structures

Hunter College, Fall 2026

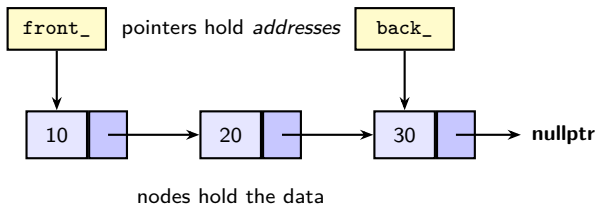
Tuesday, October 6, 2026

What You Asked

Thank you for the thoughtful questions. Before starting recursion, we answer these:

- ❶ In `if (back_ == nullptr)`, is `back_` a node or a pointer?
- ❷ Why do we assign `nullptr` to *two* pointers when the queue is empty?
- ❸ Why are the operations called `enqueue` and `dequeue`?
- ❹ Does a queue use nodes, or is it an array used with the queue idea?
- ❺ How should the circular-buffer table look (Question 2a)?
- ❻ How do I find the “index used”?
- ❼ Question 2b: why can `front_ == back_` mean both empty and full?
- ❽ Can you list Valgrind-style errors to watch for?
- ❾ Where would someone use nodes instead of the other tools we know?

Q1. Is back_ a Node or a Pointer?



- `back_` is a **pointer** (type `Node*`). It stores the *address* of the last node; it is not the node.
- `back_ == nullptr` asks “does `back_` point to anything?” If not, there is no last node: the queue is empty.
- `back_->next` means “follow the address to the node, then read its next field.”

Analogy

`back_` is a sticky note with the address of the last person in line. The note is not the person.

Q2. Two Pointers, Two Assignments

front_



nullptr

back_



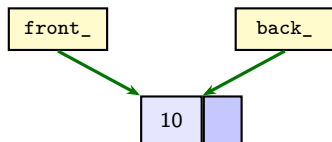
nullptr

Empty queue. No first node and no last node.

```
Node* old = front_;  
front_ = front_>next; // nullptr automatically if that was the last node  
delete old;  
size_--;  
if (front_ == nullptr) { // the queue just became empty  
    back_ = nullptr;    // second assignment: otherwise back_ dangles  
}
```

Invariant: `front_ == nullptr` $\iff$ `back_ == nullptr`
 $\iff$ `size_ == 0`. Using a stale `back_` to access freed memory
can produce an invalid read/write report (Class 10, Q1).

Q2. Two Pointers, Two Assignments

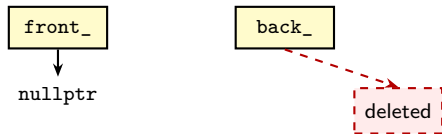


enqueue(10). The new node is both first and last: two assignments.

```
Node* old = front_;
front_ = front_>next; // nullptr automatically if that was the last node
delete old;
size--;
if (front_ == nullptr) { // the queue just became empty
    back_ = nullptr;    // second assignment: otherwise back_ dangles
}
```

Invariant: $\text{front_} == \text{nullptr} \iff \text{back_} == \text{nullptr}$
 $\iff \text{size_} == 0$. Using a stale `back_` to access freed memory can produce an invalid read/write report (Class 10, Q1).

Q2. Two Pointers, Two Assignments



dequeue() the last node. `front_` became `nullptr` on its own, but `back_` still holds the address of the freed node.

```
Node* old = front_;
front_ = front_>next; // nullptr automatically if that was the last node
delete old;
size--;
if (front_ == nullptr) { // the queue just became empty
    back_ = nullptr;    // second assignment: otherwise back_ dangles
}
```

Invariant: `front_ == nullptr` $\iff$ `back_ == nullptr`
 $\iff$ `size_ == 0`. Using a stale `back_` to access freed memory can produce an invalid read/write report (Class 10, Q1).

Q2. Two Pointers, Two Assignments

front_



nullptr

back_



nullptr

Fix: set back_ = nullptr yourself.

```
Node* old = front_;
front_ = front_>next; // nullptr automatically if that was the last node
delete old;
size--;
if (front_ == nullptr) { // the queue just became empty
    back_ = nullptr;    // second assignment: otherwise back_ dangles
}
```

Invariant: $\text{front_} == \text{nullptr} \iff \text{back_} == \text{nullptr}$
 $\iff \text{size_} == 0$. Using a stale back_ to access freed memory
can produce an invalid read/write report (Class 10, Q1).

Q3. Why “Enqueue” and “Dequeue”?

- **enqueue** = put *into* the queue (at the back). **dequeue** = take *out of* the queue (from the front).
- The names say which end is used. `add` or `insert` would not specify an end; a vector, for example, can insert at different positions.
- Different names also remind you the rule differs from a stack: first in, first out (FIFO).

	Add	Remove
Stack	push	pop
Queue	enqueue	dequeue
Vector	push_back	pop_back

Analogy

A deli counter: you take a ticket and *join the back* (enqueue); the clerk serves whoever is *at the front* (dequeue). No cutting into the middle.

Q4. Does a Queue Use Nodes or an Array?

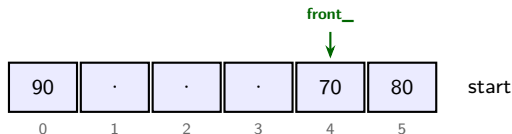
Either. A queue is a *contract* (an ADT): enqueue, dequeue, front, empty, size, first in first out. How the items are stored is a separate choice, and we have seen both.

	Linked queue	Circular array
Storage	one Node per item, linked by pointers	one fixed array data_
Remembers	front_, back_ (pointers)	front_, size_ (one design; Code 6 uses back_)
Capacity	grows until memory runs out	fixed at CAPACITY
Allocation	one new per enqueue	none per enqueue
Typical bug	stale back_	wrong % arithmetic
Speed	all operations $O(1)$	all operations $O(1)$

Q5. The Circular-Buffer Table (Question 2a)

CAPACITY = 6. Start: `front_` = 4, `size_` = 3, holding 70, 80, 90.

Operation	Index used	<code>front_</code>	<code>size_</code>	Contents (<code>front</code> first)
(start)	—	4	3	70, 80, 90
<code>enqueue(100)</code>	1	4	4	70, 80, 90, 100
<code>dequeue()</code>	4	5	3	80, 90, 100
<code>dequeue()</code>	5	0	2	90, 100
<code>enqueue(110)</code>	2	0	3	90, 100, 110

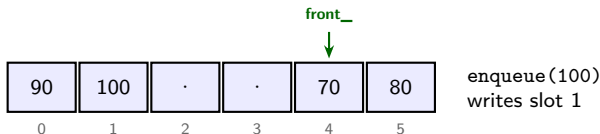


Grey values are old data left behind after a dequeue; they no longer belong to the queue.

Q5. The Circular-Buffer Table (Question 2a)

CAPACITY = 6. Start: `front_ = 4`, `size_ = 3`, holding 70, 80, 90.

Operation	Index used	<code>front_</code>	<code>size_</code>	Contents (<code>front</code> first)
(start)	—	4	3	70, 80, 90
<code>enqueue(100)</code>	1	4	4	70, 80, 90, 100
<code>dequeue()</code>	4	5	3	80, 90, 100
<code>dequeue()</code>	5	0	2	90, 100
<code>enqueue(110)</code>	2	0	3	90, 100, 110

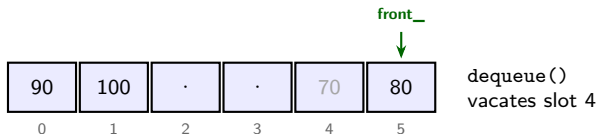


Grey values are old data left behind after a dequeue; they no longer belong to the queue.

Q5. The Circular-Buffer Table (Question 2a)

CAPACITY = 6. Start: `front_ = 4`, `size_ = 3`, holding 70, 80, 90.

Operation	Index used	<code>front_</code>	<code>size_</code>	Contents (<code>front</code> first)
(start)	—	4	3	70, 80, 90
<code>enqueue(100)</code>	1	4	4	70, 80, 90, 100
<code>dequeue()</code>	4	5	3	80, 90, 100
<code>dequeue()</code>	5	0	2	90, 100
<code>enqueue(110)</code>	2	0	3	90, 100, 110

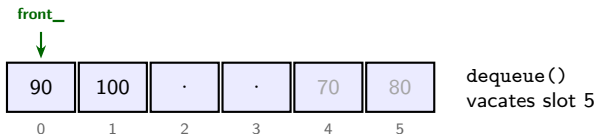


Grey values are old data left behind after a `dequeue`; they no longer belong to the queue.

Q5. The Circular-Buffer Table (Question 2a)

CAPACITY = 6. Start: `front_ = 4`, `size_ = 3`, holding 70, 80, 90.

Operation	Index used	<code>front_</code>	<code>size_</code>	Contents (<code>front</code> first)
(start)	—	4	3	70, 80, 90
<code>enqueue(100)</code>	1	4	4	70, 80, 90, 100
<code>dequeue()</code>	4	5	3	80, 90, 100
<code>dequeue()</code>	5	0	2	90, 100
<code>enqueue(110)</code>	2	0	3	90, 100, 110

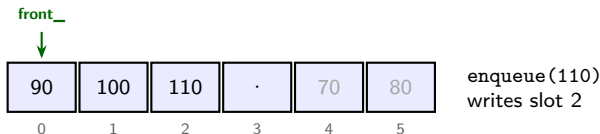


Grey values are old data left behind after a dequeue; they no longer belong to the queue.

Q5. The Circular-Buffer Table (Question 2a)

CAPACITY = 6. Start: `front_ = 4`, `size_ = 3`, holding 70, 80, 90.

Operation	Index used	<code>front_</code>	<code>size_</code>	Contents (front first)
(start)	—	4	3	70, 80, 90
<code>enqueue(100)</code>	1	4	4	70, 80, 90, 100
<code>dequeue()</code>	4	5	3	80, 90, 100
<code>dequeue()</code>	5	0	2	90, 100
<code>enqueue(110)</code>	2	0	3	90, 100, 110



Grey values are old data left behind after a dequeue; they no longer belong to the queue.

Q6. How Do I Find the “Index Used”?

enqueue: $\text{index} = (\text{front_} + \text{size_}) \% \text{CAPACITY}$

Why: the items fill slots front_ , $\text{front_}+1, \dots, \text{front_}+\text{size_}-1$. The next free slot is $\text{front_}+\text{size_}$; the % wraps it around the end.

dequeue: the slot *vacated* is the current front_ . Then $\text{front_} = (\text{front_} + 1) \% \text{CAPACITY}$.

Worked example

$\text{front_} = 4, \text{size_} = 3,$
 $\text{CAPACITY} = 6.$

$4 + 3 = 7$, and $7 \bmod 6 = 1$.
Walk 3 slots from slot 4: slot 5, slot 0, slot 1. Slot 1 is free.

Clock analogy

On a 6-hour clock, 4 o'clock plus 3 hours is 1 o'clock.
Same wrap-around.

Code 1: Queue.hpp and the Constructor

```
class Queue {
public:
    Queue();
    bool enqueue(int value);
    bool dequeue();
    int top() const; // !empty()
    int size() const;
    bool empty() const;
private:
    const static int CAPACITY = 4;
    int data_[CAPACITY];
    int front_; // index of oldest item
    int size_;  // how many items
};
```

```
Queue::Queue() : front_(-1), size_(0) {}
```

Only two integers describe the queue: `front_` and `size_`. There is no `back_`. An empty queue has `front_ = -1` (“no front yet”).

Code 2: enqueue

```
bool Queue::enqueue(int value) {  
    if (size_ == CAPACITY) {  
        return false;           // 1  
    }  
    if (front_ == -1) {          // 2  
        front_ = 0;  
    }  
    int insertIdx =  
        (front_ + size_) % CAPACITY; // 3  
    data_[insertIdx] = value; // 4  
    size_++;                  // 5  
    return true;  
}
```

- 1 Full: refuse the insertion. A full queue never reaches the index formula.
- 2 Empty before the insertion: set `front_ = 0`, so `front_` is a valid array index.
- 3 Find the insertion index: `(front_ + size_) % CAPACITY`; the `%` handles wrap-around.
- 4 Store the value.
- 5 Increment `size_`.

Code 3: dequeue, top, size, empty

```
bool Queue::dequeue() {  
    if (size_ == 0) { return false; }  
    size_--;  
    if (size_ == 0) {    // now empty  
        front_ = -1;    // back to start  
    } else {  
        front_ = (front_ + 1) % CAPACITY;  
    }  
    return true;  
}  
  
int Queue::top() const { return data_[front_]; }  
int Queue::size() const { return size_; }  
bool Queue::empty() const { return size_ == 0; }
```

- dequeue moves front_ forward by one, wrapping with %. It does not erase the old value.
- When the last item leaves, front_ resets to -1, so the next enqueue starts again at slot 0.
- empty() tests size_, never the indices (Q7).

Code 4: Watch It Wrap Around (CAPACITY = 4)

Call	index	front_	size_	slots 0 1 2 3	returns
(start)	—	-1	0	- - - -	
enqueue(1)	0	0	1	1 - - -	true
enqueue(2)	1	0	2	1 2 - -	true
enqueue(3)	2	0	3	1 2 3 -	true
enqueue(4)	3	0	4	1 2 3 4	true
enqueue(5)	—	0	4	1 2 3 4	false (full)
dequeue()	0	1	3	1 2 3 4	true
dequeue()	1	2	2	1 2 3 4	true
enqueue(7)	$(2 + 2) \% 4 = 0$	2	3	7 2 3 4	true

Read the last row

The queue holds **3, 4, 7** (front to back): slots 2, 3, then wrapping to 0. Gray values are stale: still in the array, but no longer part of the queue. `top()` returns 3.

Code 5: Two Designs, Side by Side

A. Reset front_ to -1

```
Queue() : front_(-1), size_(0) {}
enqueue(value):
    if (size_ == CAPACITY)
        return false;
    if (front_ == -1)
        front_ = 0;
    int i = (front_ + size_)
        % CAPACITY;
    data_[i] = value;
    size_++;
dequeue():
    if (size_ == 0) return false;
    size--;
    if (size_ == 0)
        front_ = -1;
    else
        front_ = (front_ + 1)
            % CAPACITY;
```

B. Leave front_ alone

```
Queue() : front_(0), size_(0) {}
enqueue(value):
    if (size_ == CAPACITY)
        return false;
    data_[(front_ + size_)
        % CAPACITY] = value;
    size_++;
dequeue():
    if (size_ == 0) return false;
    front_ = (front_ + 1)
        % CAPACITY;
    size--;
```

- **Same behavior:** identical QueueTest output; empty() is size_ == 0 in both.
- **A** uses front_ = -1 for an empty queue, so enqueue first sets front_ = 0 when inserting the first item.
- **B** is shorter and matches the Q5 table: front_ just keeps moving.

Code 6: Another Design, front_ and back_

```
Queue() : front_(-1), back_(-1) {}
enqueue(value):
    if ((back_ + 1) % CAPACITY
        == front_)
        return false;    // full
    back_ = (back_ + 1) % CAPACITY;
    data_[back_] = value;
    if (front_ == -1)
        front_ = 0;
dequeue():
    if (empty()) return false;
    if (front_ == back_)
        front_ = back_ - 1;
    else
        front_ = (front_ + 1)
            % CAPACITY;
```

```
empty():
    return front_ == -1
        && back_ == -1;
size():
    if (front_ == -1)
        return 0;
    if (back_ >= front_)
        return back_ - front_ + 1;
    // wrapped
    return CAPACITY
        - (front_ - back_ - 1);
```

- back_ is the index of the *last item*, so enqueue moves it first, then writes.
- Empty is front_ = back_ = -1, so front_ == back_ (not -1) means *exactly one item*. Full is (back_+1) % CAPACITY == front_: all 4 slots are used.
- This is another way to avoid the ambiguity from Q7: back_ means the *last item*, and -1 is reserved for empty.

Code 7: Trace of front_ and back_ (CAPACITY = 4)

Call	front_	back_	size()	slots 0 1 2 3
(start)	-1	-1	0	- - - -
enqueue(1)	0	0	1	1 - - -
enqueue(2), (3)	0	2	3	1 2 3 -
enqueue(4)	0	3	4	1 2 3 4
enqueue(5)	0	3	4	full: returns false
dequeue(), twice	2	3	2	1 2 3 4
enqueue(7)	2	0	3	7 2 3 4
dequeue()	3	0	2	7 2 3 4
dequeue()	0	0	1	7 2 3 4

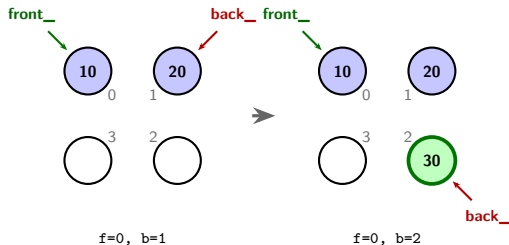
Two rows to study

enqueue(7): back_ wraps from 3 to $(3 + 1) \% 4 = 0$, so back_ < front_ and size() = $4 - (2 - 0 - 1) = 3$.

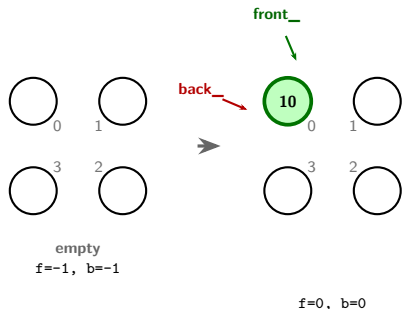
In the last row front_ = back_ = 0: *one* item (the 7), not empty. One more dequeue() resets both to -1.

Enqueue 1: Room at the Back, and the First Item

green = front_, red = back_ (last item). Blue = in the queue; gray = old value left behind.



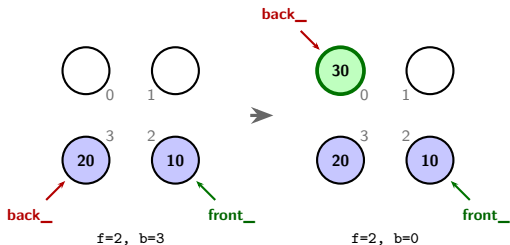
enqueue(30): $\text{back_} = (1+1)\%4 = 2$. Write $\text{data_}[2] = 30$. front_ does not move.



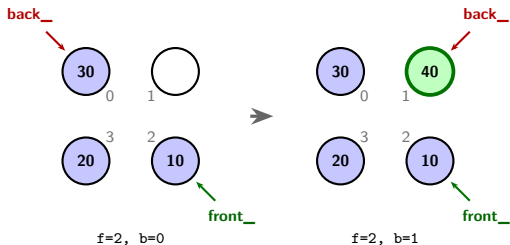
Empty queue: back_ becomes 0 and front_ is set to 0. One item, so $\text{front_} == \text{back_}$.

Enqueue 2: back_ Wraps Around the End

green = front_, red = back_ (last item). Blue = in the queue; gray = old value left behind.



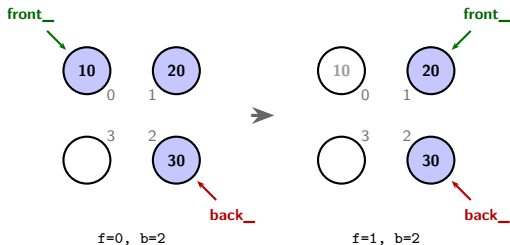
`enqueue(30): back_ = (3+1)%4 = 0`. The queue becomes **wrapped**: `back_ < front_`.



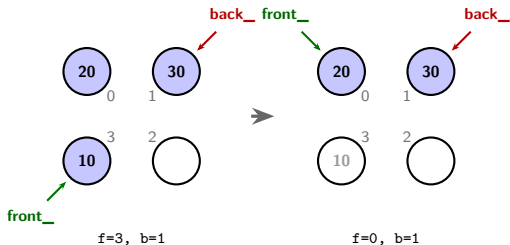
Already wrapped: same rule.
`back_ = (0+1)%4 = 1`. Now `(1+1)%4 == front_`: **full**; the next `enqueue` returns false.

Deque 1: front_ Moves Forward

green = front_, red = back_ (last item). Blue = in the queue; gray = old value left behind.

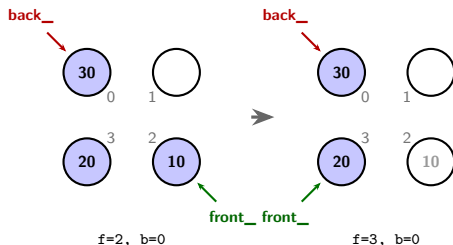
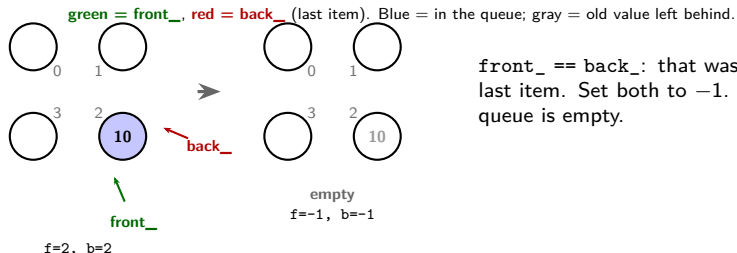


$\text{front_} = (0+1)\%4 = 1$. The 10 stays in slot 0 but is no longer in the queue.



Wrapped queue: $\text{front_} = (3+1)\%4 = 0$. Now $\text{back_} \geq \text{front_}$: not wrapped any more.

Deque 2: The Last Item, and a Wrapped Queue



$\text{front_} = (2+1)\%4 = 3$. Still wrapped ($\text{back_} = 0 < \text{front_} = 3$); items are 20, then 30.

Wrapped or not, the code is the same: enqueue moves back_, dequeue moves

front_, both with $(x+1) \% \text{CAPACITY}$.

Key Points: the front_ and back_ Circular Queue

A circular array with CAPACITY slots, indexed $0, \dots, \text{CAPACITY} - 1$.

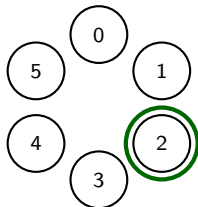
- `front_`: index of the **first** element. `back_`: index of the **last** element.

State	Test
Empty	<code>front_ == -1 and back_ == -1</code>
One element	<code>front_ == back_ and front_ >= 0</code>
Full	<code>(back_ + 1) % CAPACITY == front_</code> (all slots used)
Size, empty	<code>front_ == -1: 0</code>
Size, non-wrapped	<code>nonempty, back_ >= front_: back_ - front_ + 1</code>
Size, wrapped	<code>back_ < front_: CAPACITY - (front_ - back_ - 1)</code>

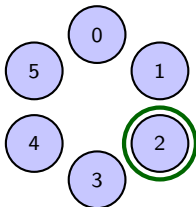
Remember

enqueue moves `back_`; dequeue moves `front_`; both use $(x + 1) \% \text{CAPACITY}$. The empty state is the only place `-1` is used.

Q7. Question 2b: Empty or Full?



Empty



Full (6 items)

Green ring: slot 2. Here `back_` is the slot *after* the last item (as in the slip). Both pictures have `front_ = 2`, `back_ = 2`.

- With no `size_`, the program has only these two integers. They are *identical* for the empty and the full queue, so `empty()` has nothing to test (the array cannot help: dequeued values stay behind).
- Analogy: two runners on a circular track at the same spot. Race just started, or one lapped the other? Positions alone cannot tell.
- Fixes: keep `size_` (our choice), keep a bool `full_`, or leave one slot empty.
- Another design (Code 6): let `back_` be the *last item* and reserve `front_ = back_ = -1` for empty. Then `front_ == back_ && front_ >= 0` means one item, and all slots are usable.

Q8a. Valgrind: Memory You Do Not Own

Valgrind says	It means	Typical cause here
Invalid read / write of size N	You touched memory you do not own.	Index out of range; stale pointer.
Address ... is 0 bytes after a block of size N alloc'd	Just past the end of an array.	Off-by-one: <code>i <= size_</code> in- stead of <code>i < size_</code> .
Address ... is ... bytes inside a block of size M free'd	You accessed memory after it was freed (dan- gling pointer).	Stale <code>back_</code> ; using <code>data_</code> after <code>delete[]</code> .

The lines under the message show the *access* (top), where the block was *freed*, and where it was *allocated*. Read all three.

Q8b. Valgrind: Uninitialised Values and Bad Frees

Valgrind says	It means	Typical cause here
Conditional jump or move depends on uninitialised value(s)	An if or loop used a value never set.	Member not initialised; <code>new int[n]</code> without <code>{}</code> .
Use of uninitialised value of size 8	Garbage read, often a pointer.	Uninitialised pointer member, then <code>*p</code> .
Invalid free() / delete / delete[]	Freed something not allocated, or freed it twice.	Shallow copy: two objects delete one array (Rule of Three).
Mismatched free() / delete / delete[]	<code>new[]</code> paired with <code>delete</code> , or the reverse.	Wrote <code>delete data_</code> instead of <code>delete[] data_</code> .

Q8c. Leaks, Limits and a Workflow

Leak categories (`-leak-check=full`)

- **Definitely lost:** no pointer to the block exists. A real leak (our `grow()` bug).
- **Indirectly lost:** reachable only through a lost block (the rest of a list whose head was lost).
- **Possibly lost:** only a pointer into the middle of the block exists.
- **Still reachable:** a pointer still exists at exit; usually less serious.

Limits: Valgrind cannot see overruns of *stack* or global arrays; try `-fsanitize=address,undefined` too.

Workflow: compile with `-g`; run `valgrind -leak-check=full ./a.out`; fix the *first* error and rerun (later ones are often side effects); `-track-origins=yes` finds where an uninitialised value came from. Goal: 0 errors and “All heap blocks were freed.”

Q9. When Do Nodes Win?

Use nodes when ...

- you insert or remove **in the middle**: $O(1)$ and nothing shifts, *after a known predecessor* in a singly linked list, or *at a known node* in a doubly linked list;
- the size is **unpredictable** and you want no big reallocate-and-copy;
- other code holds **pointers to elements** that must stay valid;
- the data is **naturally linked**: trees (BST, 11/19), tries (11/24), hash chains (12/1), graphs.

Everyday examples: a music playlist (“play next”), undo/redo history, browser back/forward, a memory allocator’s free list, an LRU cache.

Honest caveat

For plain storage, `std::vector` is often *faster* in practice (contiguous memory). Choose nodes for a reason above, not by default.

Quick Check

- 1 A linked queue holds one node. Suppose you `dequeue()` it but forget to reset `back_` to `nullptr`. Which pointer dangles, and what can Valgrind report if you enqueue again?
- 2 `CAPACITY = 8, front_ = 6, size_ = 4`. At which index does enqueue write?
- 3 You wrote `delete data_;` for an array made with `new int[n]`. What does Valgrind call this?

Next: recursion and the call stack.

Quick Check

- ❶ A linked queue holds one node. Suppose you `dequeue()` it but forget to reset `back_` to `nullptr`. Which pointer dangles, and what can Valgrind report if you enqueue again?
- ❷ `CAPACITY = 8`, `front_ = 6`, `size_ = 4`. At which index does enqueue write?
- ❸ You wrote `delete data_;` for an array made with `new int[n]`. What does Valgrind call this?

Answers

1. `back_` dangles. Using it in the next enqueue is an invalid access to freed memory (for example, an “Invalid write”).
2. $(6 + 4) \bmod 8 = 2$.
3. Mismatched `free()` / `delete` / `delete[]`.

Next: recursion and the call stack.